3Laws

Release pre-release

Thomas Gurriet (tgurriet@3laws.io)

Mar 07, 2024

1 Introduction **3**
1.1 Architecture . . . 3
1.2 Supervisor interfaces . . . 4
1.3 RunTime Assurance Module (RAM) . . . 4
1.4 Robot Diagnostic Module (RDM) . . . 5
1.5 Configuration . . . 5

2 Getting Started **7**
2.1 1. Installation . . . 7
2.2 2. Configuration . . . 8
2.3 3. Launch . . . 17
2.4 4. Operations . . . 17

3 Installation and Update **19**
3.1 Quick Installation . . . 19
3.2 Non Interactive Installation . . . 19
3.3 Update . . . 20

4 Control Panel **21**
4.1 Launch the Control Panel . . . 21
4.2 Use the Control Panel . . . 21

5 Visualization **25**
5.1 Dashboards . . . 25
5.2 Navigation Tools . . . 26
5.3 Incident Management . . . 26

The 3Laws Supervisor is a tool that runs partly on your robot(s) and partly as a cloud service.

Chapter 1

Introduction

The 3Laws Robotics Supervisor is a software package that provides a control filter to ensure safety without compromising performance. Additionally, the Supervisor contains a diagnostic module that monitors multiple metrics that indicate safety and performance.

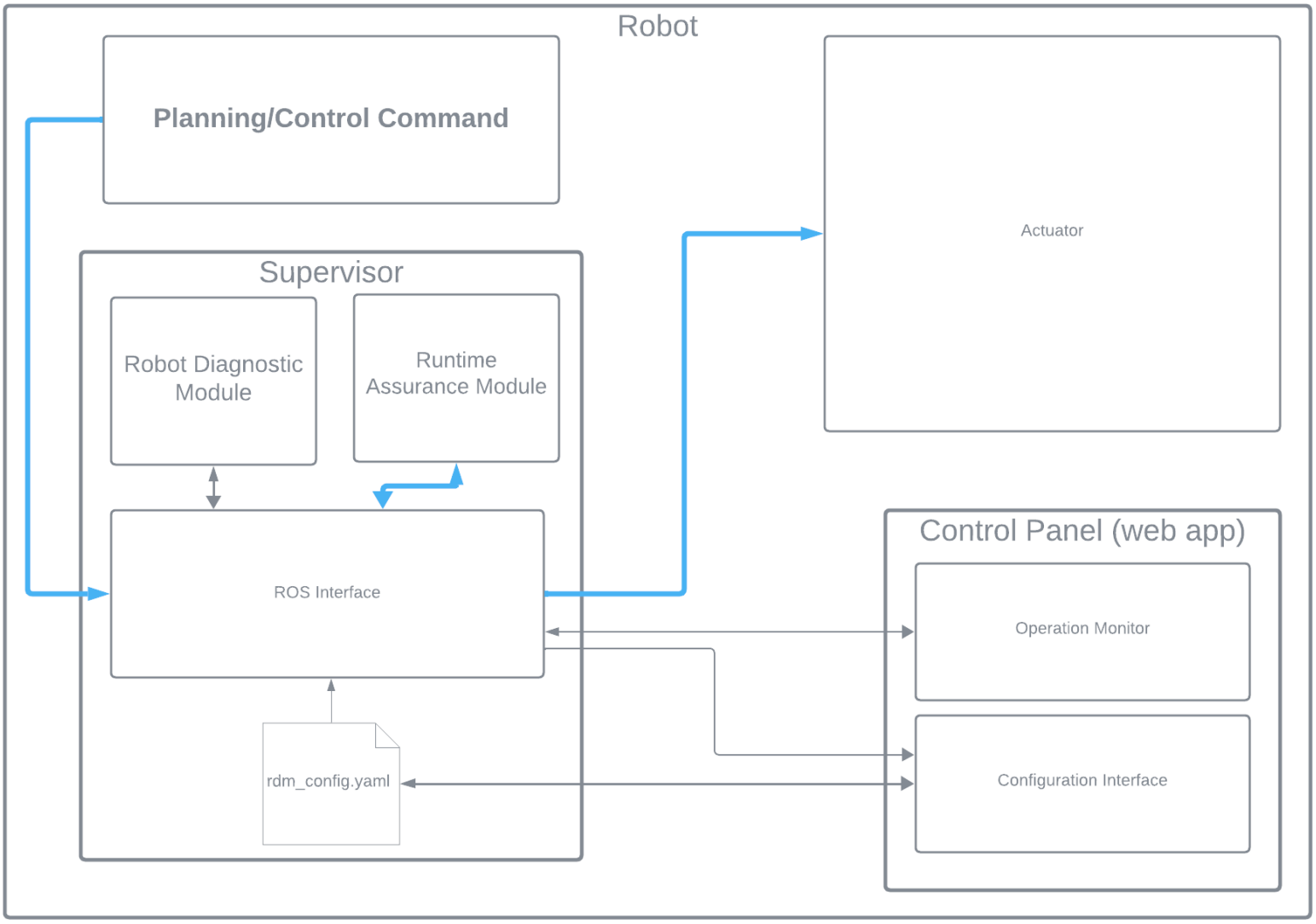
Multiple ROS version are supported:

Ubuntu Distribution	ROS1 version	ROS2 version
22.04	N/A	Humble/Iron
20.04	Noetic	Galactic/Foxy
18.04	Melodic	N/A

1.1 Architecture

The Supervisor is composed of 3 main modules:

- **RAM**: The Run-time Assurance Module is the core of the Supervisor. It is a filter that operates at the controller level to ensures that the directional commands sent to the robot do not violate proximity constraints relative to laserscan data. This component runs in near-real-time on the robot.
- **RDM**: The Robot Diagnostic Module is a collection of metrics calculated to indicate the robot health and safety. Data is aggregated on the robot by this module and is then shared to a cloud server for display.
- **Control Panel**: The Control Panel is an optional web-based application that guides the user through the configuration of the Supervisor. Based on the data provided by the user, it creates (or updates) a configuration file used by the Supervisor's various capabilities. The Control Panel has some abilities to visualize the robot's safety. This component does not need to be active (or running) during operation of the robot, except as desired for visualizing the metrics available through it. Once the Supervisor is configured through the Control Panel, turning off the lll_control_panel.service is a reasonable step.



1.2 Supervisor interfaces

The 3Laws Robotics Supervisor currently requires that the robot is using the Robot Operating System (ROS). The Supervisor is a ROS node that subscribes to the robot's state, sensors, planning, computational measures, perception, and control commands. If the RTA capability is enabled, Supervisor will also publish safety-filtered control commands to the robot's actuators.

1.3 RunTime Assurance Module (RAM)

The RunTime Assurance Module (RAM), also referred to as **Copilot**, is a filter that operates at the control level. It is designed to ensure that the robot's control commands are safe and do not violate safety constraints related to proximity to points measured by a laser scanner. Based on formal mathematical proven methods, RAM is able to prevent the robot from colliding while still allowing the robot to reach maximum performance when the system is far from any obstacles in its current travel direction.

This ability allows development of the robot's control and planning algorithms without worrying

about collision avoidance.

RAM uses basic kinematic and dynamic models for the robot in order to predict potential collisions. Supervisor currently supports differential-drive (able to rotate-in-place and translate), single-track steered, and omni-directional (able to translate sideways in addition to rotations and forward/back motion) vehicles.

1.4 Robot Diagnostic Module (RDM)

The Robot Diagnostic Module (RDM) is a diagnostic tool that monitors and compute metrics about the robot health and safety. The computed metrics are published on ROS topics that. High frequency versions of the metrics can be used to monitor the robot's health and safety in real-time. In other words, these metrics are available to other processes on the robot, that can then use the information to make decisions.

The metrics are also summarized and sent to a cloud database for display and analysis. 3LawsRobotics cloud dashboards help diagnose issues the robot might have. These Dashboards are made using Grafana.

1.5 Configuration

For effective operation, the Supervisor needs to be configured. The Control Panel will help with Supervisor configuration. The Control Panel will also help visualize operation of the Supervisor's Run-time Assurance (RTA) in action.

Chapter 2

Getting Started

This guide will help you install and configure the 3Laws Supervisor on your robot.

3Laws Supervisor is designed for installation on Ubuntu systems with ROS1 or ROS2 already deployed on the computer. The Supervisor is roughly made up of 3 functionalities that are discussed in this manual:

1. Robot diagnostics monitoring (RDM) collects metrics about the system operation in real time. Some of the threshold violations are published in real-time locally, and some are published only in aggregated fashion. The published messages can be sent to a cloud-based dashboard server or can be used locally for decision making by customer software.
2. Run-time assurance (RTA) or "Copilot" is a "safety filter" that takes the information about collision threshold violations from the diagnostics monitor and produces corrective actions to keep the robot away from the possible collision. By default, this capability is no active.
3. Control Panel: A configuration graphical (browser-based) tool is available to help set the Supervisor up for the robot.

Table of Contents

2.1 1. Installation 7
2.2 2. Configuration 8
2.3 3. Launch. 17
2.4 4. Operations 17

2.1 1. Installation

To install Supervisor on your system, open a terminal and run the following command:

```
bash <(curl
https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh)
```

This script will download a package from github and will begin the installation.Note that the package name starts with "lll-rdm". The "lll" represents 3Laws. The product's original name was Robot Diagnostics Module. As the script proceeds it will try to determine your system's configuration so that the appropriate components are installed. The script will prompt you for help with configuration if it is unable to find your computer's ROS distribution or architecture.

During the execution of the script, several questions will be asked: #. A request to confirm the desire to download the package.

1. Confirmation of the desire to install the package.
2. A *sudo* password to be able to install to shared folders (/opt/ros).
3. Confirmation of the desire to *download* the optional Control Panel component which is designed to make the configuration process easier.

4. Confirmation of the desire to *install* the optional Control Panel component.

The script will add components to the global ROS installation. These new components will not be available until the ROS paths are updated. If your configuration automatically runs the ROS setup script when a new shell is started, please close the terminal and open a new one. Otherwise, please run the following command:

```
source /opt/ros/<DISTRO>/setup.sh
```

The Supervisor will now be available for operation.

2.2 2. Configuration

The 3Laws Supervisor is able to support several different types of robots and it needs to connect to your system's data sources and sinks. The configuration process aims to specify this type of information.

A graphical (browser-based) configuration tool (called **Control Panel**) is available to help with the configuration effort.

After installation open a browser window and navigate to the address *http://localhost:8000*. If this window does not appear, please manually start the Control Panel server with the command:

```
sudo systemctl start lll_control_panel
```

The Control Panel's capabilities can be augmented through a rosbridge server. With that server, the control panel can more easily display some real-time information about the Supervisor's status. To install and start a rosbridge server (where <rosdistro> is replaced with the version of ROS on your system):

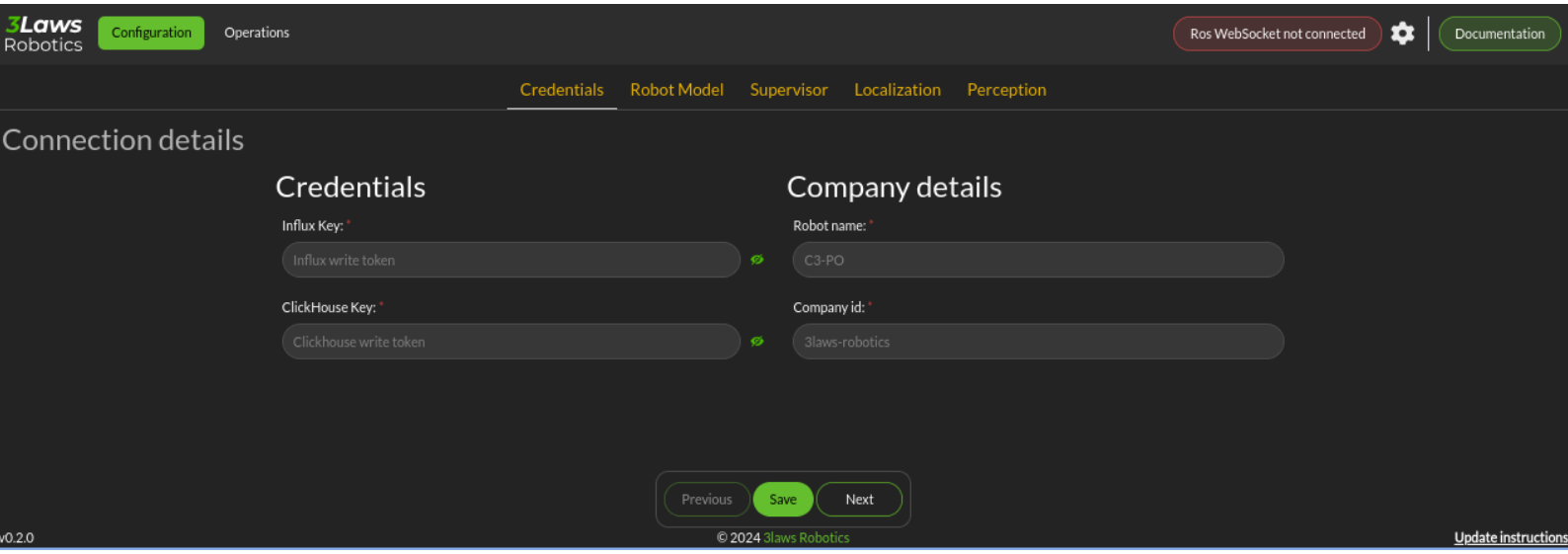
```
sudo apt-get install ros-<rosdistro>-rosbridge-server
ros2 run rosbridge_server rosbridge_websocket
```

This will provide a websocket server at *ws://localhost:9090* that the control panel can connect to in order to retrieve topics and services information.

The initial of the Control Panel is the "Configuration" page, which consists of sections (tabs) listed as *Credentials*, *Robot Model*, *Supervisor*, *Localization*, and *Perception*.

2.2.1 Credentials

The *Credentials* page is displayed here:



The 3Laws Supervisor is designed to work in conjunction with a cloud-based server. When you purchase the Supervisor, 3Laws will provide you with the credentials to connect to the web-based server and with the official company name that is used for your serivces.

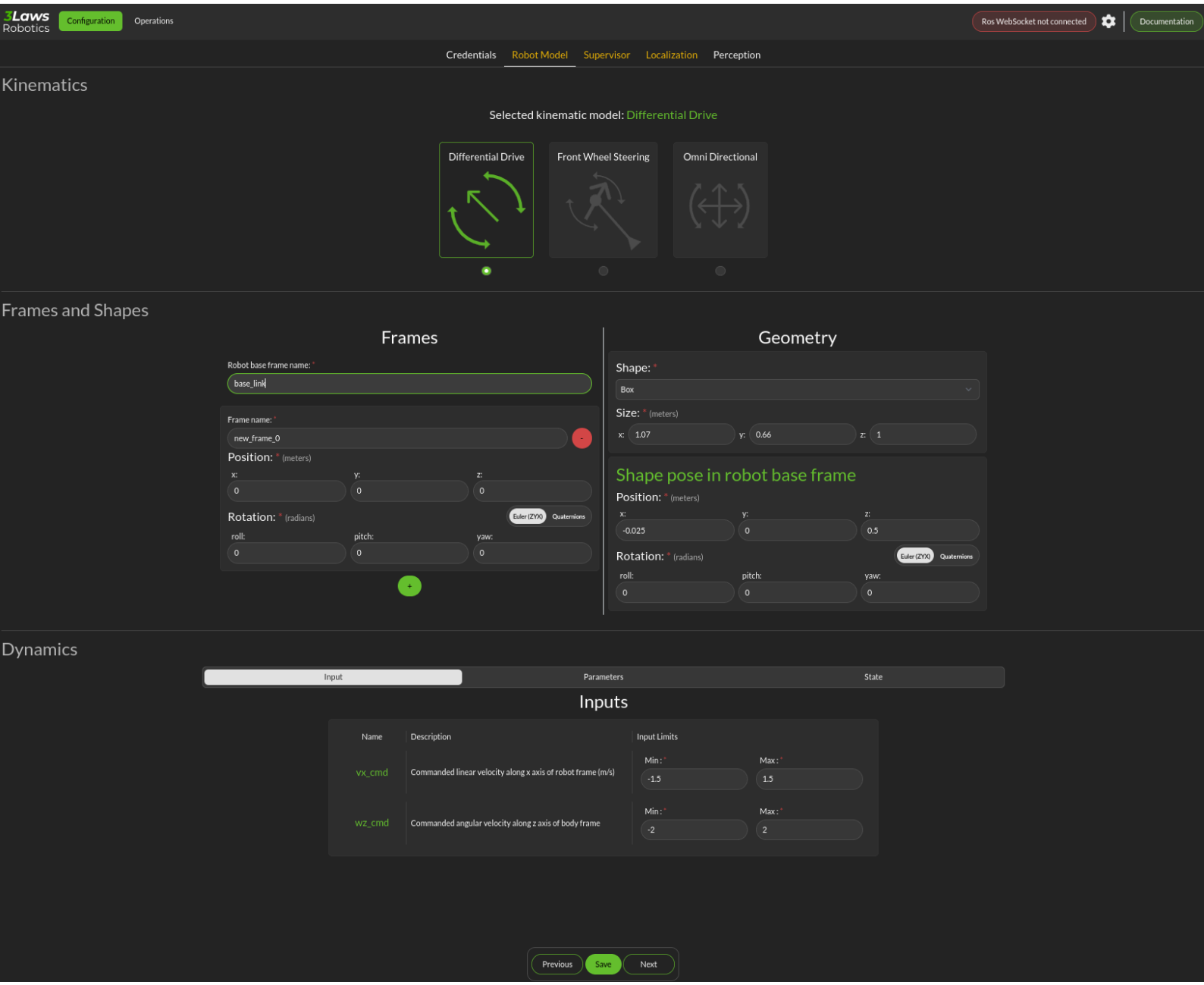
- **Credentials**: 3Laws will provide you with an Influx Key and a ClickHouse key so that your robot can connect to the cloud-based server and can send its summarized information for display through Grafana.
- **Robot Name**: This identifier will be different for each robot where you install Supervisor. It should contain a name you can use to identify an individual robot.
- **Company ID**: As part of the registration process, your company's name will be used to create an account on the cloud server. Please make sure that the name matches what 3Laws used to create your account. This name is used for logging into the cloud server.

The upper-right of the display contains the Rosbridge connection status (upper right), configuration for the Rosbridge (gear next to the status), and a link to this documentation. There is an "Update Instructions" link in the lower right that brings up a reminder of the instruction to run the installer script.

The **Save** button on each page of the Control Panel should be pressed before moving on to another page.

2.2.2 Robot Model

The Configuration page for the Robot Model is where the robot's geometry and kinematics are specified.



Throughout this documentation, a red asterisk (*) indicates a *required* field.

- **Robot Type**: Supervisor currently supports differential drive, front-steered, and omni-directional mobile platforms. A differential drive vehicle can rotate around a fixed location by driving one wheel forward and its pair in reverse. It can move forward or backwards by driving both wheels in the same direction. A front-steered vehicle operates like a car or bicycle. It can move forwards or backwards, but direction is controlled by steering, and the vehicle has a fixed wheelbase distance between the forward axle and the rear axle. An omni-directional robot is usually implemented using wheels that have a series of smaller wheels mounted at 45 degrees on the outer rim. By moving one axle forward and others backwards, the vehicle can move sideways. Rotation can be effected by moving the wheels on one side forwards and the wheels on the other side in reverse.
- **Frames and Shapes**: The vehicle's reference frames and geometry are specified in this section.
    - Frames: The name of the robot base frame must be specified here. Additional static frames can be defined by clicking the + icon. The additional frames are static, and will follow the motion of the base robot frame.

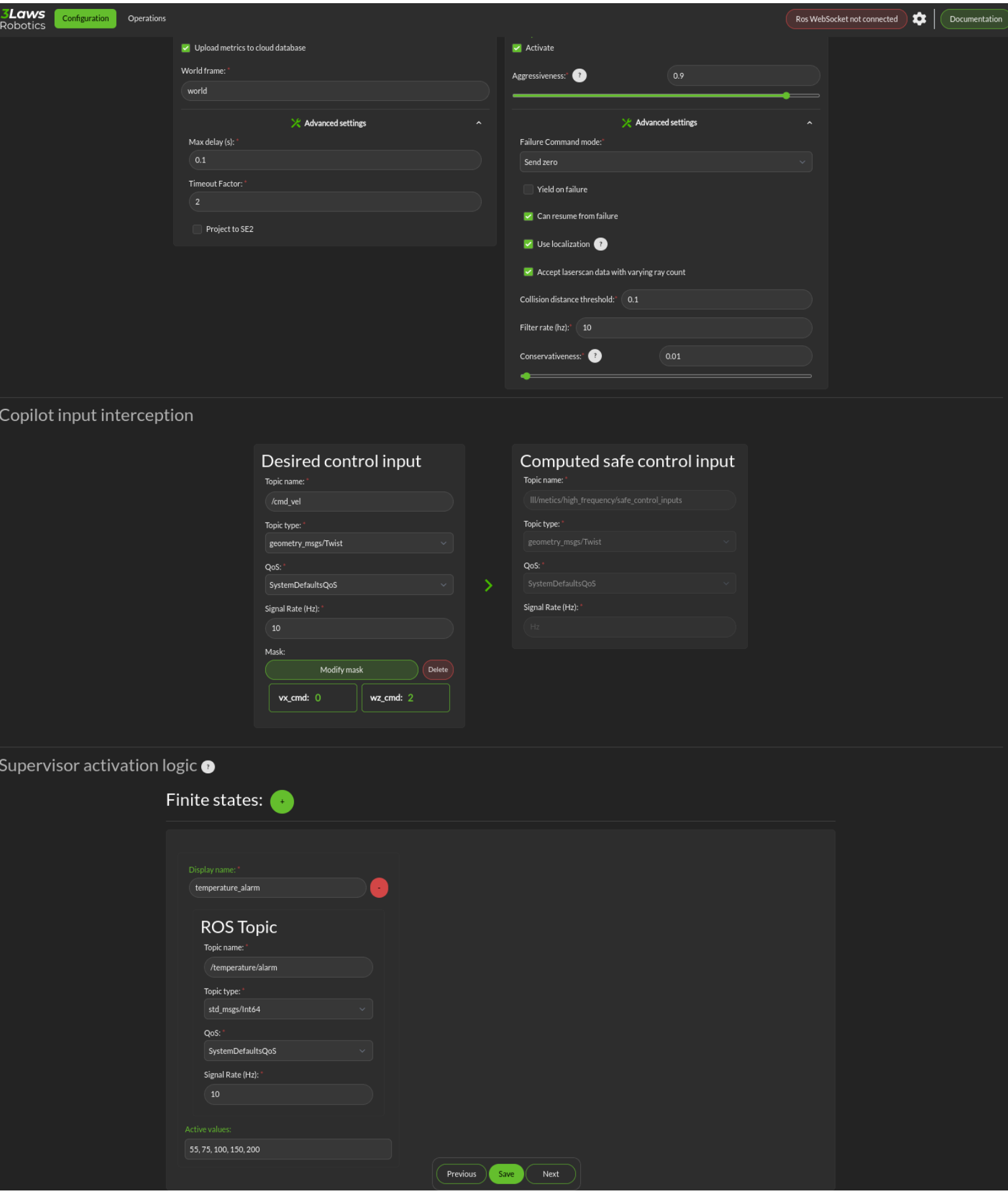
- Shape: The robot's shape is used in order to calculate the distance between the outer boundaries of the robot body and any scan points. Basic shapes that are currently supported include sphere, box, capsule, point, cone and cylinder. The size entries change based on the selected shape. Cylinders and capsules both require length and radius. The difference is that a capsule will have hemispheres on the ends while the cylinder ends are flat. A box frame consists of x-length, y-length, and z-length.
    - Shape Pose in Robot Frame: The relative orientation of the robot's shape with respect to the base frame needs to be specified. Note that the rotation can be specified either using quaternions or Euler angles. The quaternion order is w, x, y, z. When Supervisor is running, it is a good idea to run rviz (or rviz2) to display the orientations and frames.

- **Dynamics**: The Dynamics section has 3 subsections: Input, Parameters, and State.
    - The Input section supports specification of maximum and minimum limits for the translational and rotational speeds at which the robot can be commanded. For monitoring, if these values are exceeded, an event to this effect will be issued. If the RTA/Copilot component is active, these limits will be applied to the filtered outputs. For steered vehicles, instead of limits on rotational speeds, steering limits are specified.
    - Parameters: The only robot type that has inputs in the parameters section is the steered robot. *wheel_dx* is the wheelbase length for the vehicle. *origin_x* is the distance from the rear axle to the robot base frame.
    - State: For all robot types, the State category provides the definitions of the variables that are considered as the "states". These definitions are important when trying to create "masks" to map between the input ROS variable types and the robot states. The first, is considered state 0 (typically x position), the second is state 1 (typically y position) and the third is state 2 (typically yaw).

Remember to save each page after inputing the data.

2.2.3 Supervisor

The Configuration > Supervisor page contains configuration entries for both the RDM (monitor) and RTA (Copilot) components.


3Laws Robotics
Configuration
Operations
Ros WebSocket not connected
Documentation
Upload metrics to cloud database
World frame:
world
Advanced settings
Max delay (s):
0.1
Timeout Factor:
2
Project to SE2
Activate
Aggressiveness:
0.9
Advanced settings
Failure Command mode:
Send zero
Yield on failure
Can resume from failure
Use localization
Accept laserscan data with varying ray count
Collision distance threshold:
0.1
Filter rate (hz):
10
Conservativeness:
0.01
Copilot input interception
Desired control input
Topic name:
/cmd_vel
Topic type:
geometry_msgs/Twist
QoS:
SystemDefaultsQoS
Signal Rate (Hz):
10
Mask:
Modify mask
Delete
vx_cmd: 0
wz_cmd: 2
Computed safe control input
Topic name:
lll/metics/high_frequency/safe_control_inputs
Topic type:
geometry_msgs/Twist
QoS:
SystemDefaultsQoS
Signal Rate (Hz):
Hz
Supervisor activation logic
Finite states:
Display name:
temperature_alarm
ROS Topic
Topic name:
/temperature/alarm
Topic type:
std_msgs/Int64
QoS:
SystemDefaultsQoS
Signal Rate (Hz):
10
Active values:
55, 75, 100, 150, 200
Previous
Save
Next

- **Basic configuration**: To enable broadcasting of the aggregated statistics to the cloud account, select the *Upload metrics to cloud database" box.
  - World Frame: Similar to "base robot frame", the name of the world frame (typically *odom* or *map*) must be specified.
    - Advanced Settings: Max delay (s) and Timeout Factor are thresholds for triggering events informing that data failed to arrive (if data is not received for max-delay * timeout-factor seconds. If the copilot (run-time assurance) is active, failure to receive robot state or desired control input (in timeout factor * 1/signal-rate) will cause the copilot to switch to the Failure Command Mode (which is explained below).
  - Copilot : The Copilot enables the run-time assurance capability where desired commands to the robot from the autonomy stack ("desired inputs") are modified in order to avoid collisions, and altered versions are published through a separate message.
    - Activate: This checkbox controls whether the run-time assurance intercepts and modifies commands from the planner/trajectory generator and forwards modified versions to the vehicle. The Copilot will only modify the outputs if the option is activated. If it is not activated, the unmodified "desired control input" will be transmitted on the designated "Computed safe control" message.
    - Aggressiveness: This parameter controls how far from the nearest obstacle the safety filter starts having more effect on the commands and how strongly the safety filter pushes the robot back into the "safe" region if the safety definition has been violated. A larger value means that the control inputs from the planner will start to be modified when the robot is farther from an object/obstacle. In general this will produce larger margins. A larger value also means that if an obstacle is detected within the collision distance, the command modified by the run-time assurance will try to move the robot away from the object more aggressively. Typical values are between 0.5 and 1.0, but values in the range of 1000 might be used in reasonable situations. A smaller value means that the the robot will get closer to the obstacles (higher performance) before being diverted.

The following are under the "Advanced Settings". If the Copilot is activated, these should be customized:

- Failure Command Mode: The run-time assurance constantly monitors to ensure that it has enough data to determine whether the robot is in a safe condition. The minimum data required is the vehicle state, the laser scan values, and the commanded/desired input. If any of these is missing the RTA can switch to the failure command mode:
  - Send Zero: In this mode the run-time assurance commands zero speed and zero turn/rotation in order to bring the vehicle to a stop.
  - Do not Publish: Another option is to stop publishing values. This option should only be used if the robot has its own mechanism to put itself in a safe condition if it is not receiving commands.
- Yield on failure: This checkbox is like the "Send Desired" option. Setting this checkbox will override the failure control mode and just forward the "Desired control input" unmodified.
- Can resume from failure: With this checkbox filled in, once the input data (control input, laser scan, and state) values start appearing after a failure, the robot will be commanded back into motion (if the desired control input is asking for that). If the box is unchecked once there is a failure, the robot will remain stopped until the Supervisor is restarted.
- Use localization: Supervisor provides a MarkerArray that displays the robot's bounding box and rays to the closest obstacles. If "Use Localization" is set,

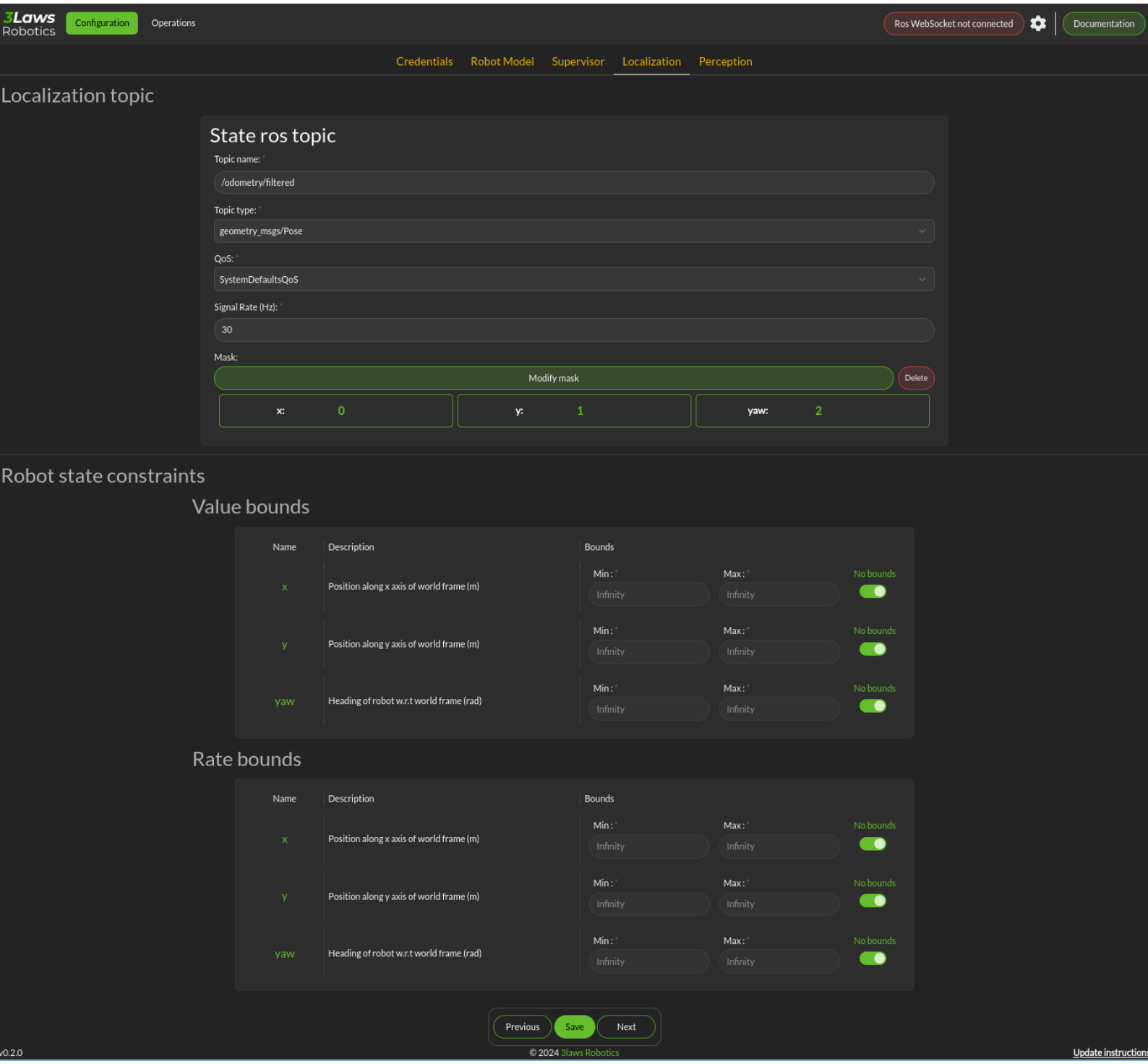
the display is created relative to the world frame. In situations where the localization may be less reliable, this checkbox can be deselected, and the visualization will be based on the current robot base frame.

- Accept wrong size laserscan: One of the checks that is made on the incoming data is that the laserscan is delivering the expected number of scan points each frame. However, there are many laser scanners that are not consistent in the number of scan points they deliver. Checking this option allows for laser scanners with non-constant number of scan points reported.
- **Collision distance threshold**: This is one of the most important values to set. This defines the distance between the edge of the robot and the nearest scan at which safety exists. If the measured distance drops below this value, the system is considered to be in an "unsafe" configuration.
- Filter rate (hz): The frequency at which the run-time assurance publishes outputs. It is recommended that the run-time assurance run at the same rate as the desired control input or at a faster rate.
- Conservativeness: A factor that specifies how much uncertainty the robot operator thinks there is in the localization and sensor data. The ratio between aggressiveness and conservativeness is the main controller of the behavior. Values below 0.1 are recommended.

- **Copilot input interception**: This section relates to republishing the control commands to the robot that are being sent from the autonomy stack. The values will be published on the *lll/metrics/high_frequency/safe_control_inputs* channel if the Copilot is activate or not. However, the values will only be different from the *Desired control input* if the Copilot is active.
  - Desired control Input: This is the commands requesting speed and rotation (or speed and steering) that the autonomy stack is publishing. The ROS message type is needed so that the RDM knows what to monitor in order to calculate the barrier function value. The message quality and receipt rate are monitored as part of the aggregated metrics, and if it fails to arrive within the expected time [1/(signal rate) * Timeout factor], an event will be created and the Copilot will transition to the failure command mode.
  - Safe Control Signal: The right side of this area is purely informational. However, if the robot is to be controlled by the run-time assurance signal, it needs to subscribe to the message that is presented here.
- **Supervisor activation logic**:
  - Finite States are messages that the Diagnostics can listen to and issue events when the value of the finite state matches a predefined value. This could be useful to provide notifications, for example, as the temperature of a component or process reaches predefined values. If the temperature goes too high or too low, the time at which the threshold is reached may be of interest. Keep in mind that if a value changes too rapidly, it might pass through a single value too quickly to be detected at that value, so in some cases it may make more sense to use an integer representation of the variable. Internally, the diagnostic converts the measurand and the threshold to strings for comparison.

2.2.4 Localization

Dialogs to connect to the state information provided for the robot and for configuring alerts based on state are on the Configuration > Localization page. contains configuration entries for both the RDM (monitor) and RTA (Copilot) components.

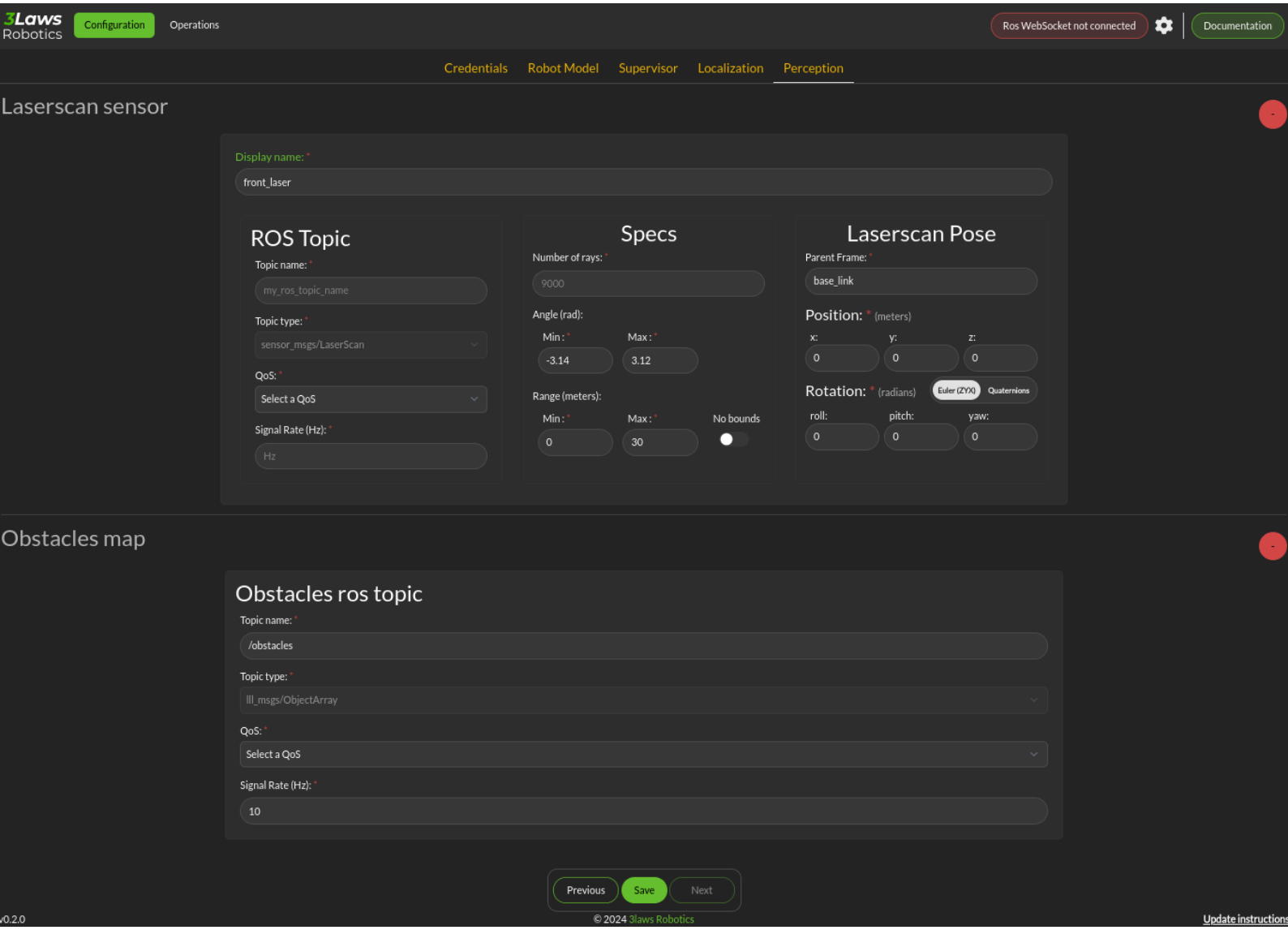


- **Localization topic**: The connection to the ROS state topic is configured in this area. As with input commands, the message topic name, message topic type, expected message topic quality, and expected message rates are specified. If the message quality fails or the message receipt rate is not met, the monitor will issue alerts, and the Copilot will switch to the Failsafe strategy. The mask needs to be customized if the localization topic is a vector of values that is not a standard ROS message. The index in the input vector relating to the individual states (x, y, yaw) nees to be set correctly.
- **Robot state constraints**: Limits on the absolute location (relative to the origin of the world frame) and limits on the measured rates of change (with respect to time) of the vehicle state are set in this area in order to trigger events and alerts for the monitoring function. The "no

bounds" option allows infinite travel in the respective directions or speeds.

2.2.5 Perception

The collision avoidance depends mostly on the Configuration > Perception dialog where the connection and description of either a laserscan sensor or an obstacle map message is configured.



- **Laserscan sensor**: The Supervisor can handle data points from one 2-dimensional laser (LIDAR) scanner.
  - Display name: A user-specified name can be provided so that diagnostic information can quickly be understood.
  - ROS Topic: The ROS message name, topic type, quality of service and Signal Rate must all be specified so that the Supervisor can subscribe to the sensor data.
  - Specs: The expected number of points per scan along with the first (typically minimum) and last (typically maximum) angle must be specified so that the angular resolution can be calculated for the nominal case. The first and last angle values should normally describe a laser that scans in the clockwise direction, so the first is smaller than the last. However, if the laser scans in the counter-clockwise direction the first angle should be set to be smaller than the last. It is very important that the total range of the laser is less than 2π. The Supervisor does not disambiguate angles if the total field is larger than 2π. The scan-

ner'srange can be set so that readings smaller than the minimum or larger than the maximum are discarded.

- Laserscan Pose: The orientation and position of the laserscan relative to the vehicle body or whichever frame is used must be specified. As with the robot's body position, the user is advised to plot the data in rviz to ensure that the geometry is set correctly.

- **Obstacle Map**: An existing perception system can be used instead of a 2D-LIDAR, but it most provide an ObjectArray that matches the definition for an lll_rsgs/ObjectArray. The definition is as follows:
  std_msgs/Header header Object[] objects

where Object[] is defined by:

std_msgs/Header header
Identifier of the object string id
Object geometry, and pose of geometry in object frame ObjectGeometry geometry
Object pose world frame geometry_msgs/PoseWithCovariance pose # Object velocity in object frame geometry_msgs/TwistWithCovariance velocity
Object behavior model ## Bounds on object frame velocity (considered inactive if non finite) geometry_msgs/Twist velocity_upper_bounds geometry_msgs/Twist velocity_lower_bounds ## Bounds on object frame velocity norms (considered inactive if strictly less than 0) float64 linear_velocity_norm2_bound float64 angular_velocity_norm2_bound ## Bounds on object frame acceleration (considered inactive if non finite) geometry_mix's/Accel acceleration_upper_bounds geometry_msgs/Accel acceleration_lower_bounds ## Bounds on object frame acceleration norms (considered inactive if strictly less than 0) float64 linear_acceleration_norm2_bound float64 angular_acceleration_norm2_bound

2.3 3. Launch

Before starting the supervisor be sure to have your ROS environment correctly set up and sourced.

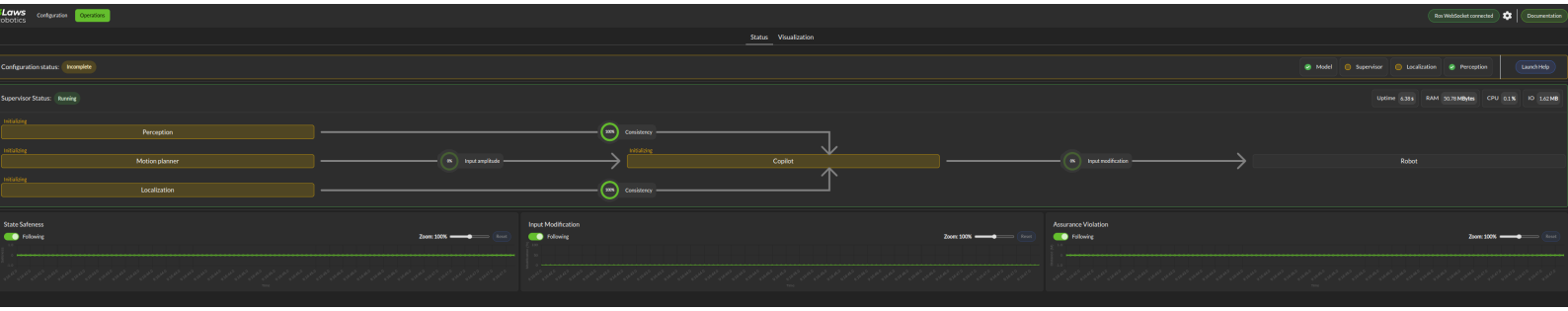
```
source /opt/ros/<DISTRO>/setup.sh
```

To launch the Supervisor, use the following command:

```
ros2 launch lll_rdm rdm.launch.py
```

2.4 4. Operations

If the websocket (rosbridge) is running along with the supervisor, the Control Panel's *Operations* tab can be used to obtain a quick overview of the status of the copilot.



In the image above, the Supervisor is operational and the Copilot is configured to be active as indicated by the arrows between them. However, these boxes are colored yellow/gold, indicating that they are still initializing. The framed section above the diagram shows the activity status for some of

the critical components:

- The model is healthy (green check).
- The Supervisor in unhealthy/initializing (gold).
- Localization is also unhealthy/initializing (gold).
- Perception is reported as healthy.

The lower section of the panel is showing strip charts. The categories that are currently displayed represent:

- the State Safeness - the barrier function value. When this value goes to zero or below zero, the system is evaluated as being in a collision state.
- the Input Modification status - When this value is zero, the copilot is not modifying the input from the autonomy stack. That is, the filtering is in passive mode. When this value is non-zero, it means that the copilot is actively modifying the commanded input.
- Assurance violation represents that during the process of solving to find the closest input to the desired one, assumptions in the model or uncertainty had to be violated in order to produce a valid solution. In this case the copilot is producing the best input to bring the system to the desired set, but the guarantees that the system is in the desired region and will remain there no longer hold.

Chapter 3

Installation and Update

Table of Contents

3.1 Quick Installation . . . 19
3.2 Non Interactive Installation . . . 19
3.3 Update . . . 20

The Supervisor is distributed as a debian package. A package is available for each of the following configuration

Ubuntu Distribution	ROS1 version	ROS2 version	Architecture
22.04	N/A	Humble/Iron	amd64/arm64v8
20.04	Noetic	Galactic/Foxy	amd64/arm64v8
18.04	Melodic	N/A	amd64/arm64v8

3.1 Quick Installation

Copy and paste the following command in a terminal to install the Supervisor:

```
bash <(curl
https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh)
```

This will install the Supervisor for the current distribution and architecture and the control panel.

At the end of the installation, the control panel will be started and you will be able to start the configuration of the supervisor. To do so open a web browser and go to the following address: http://localhost:8000

3.2 Non Interactive Installation

If you want to install a specific package in a non interactive mode, here is the process:

- Download the script:

```
wget https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh
```

- Make it executable

```
chmod +x install.sh
```

- Run it with your arguments

```
sudo ./install.sh [-hyf] [-r <ROS_DISTRO>] [-a <ARCH>] [-v <UBUNTU_VERSION>]
```

if `-yf -r <ROS_DISTRO> -a <ARCH> -v <UBUNTU_VERSION>` specified, the script is non interactive

3.3 Update

To update the Supervisor, you can use the same command as for the installation.

The supervisor will be updated to the latest version available for your distribution, **All your configuration will be kept**.

```
bash <(curl
https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh)
```

Chapter 4

Control Panel

Table of Contents

4.1 Launch the Control Panel 21
4.2 Use the Control Panel 21

The **Control Panel of 3LawsRobotics** is a helper tool to help the configuration of the 3Laws Robotics supervisor. It provides a web interface to configure the 3Laws Robotics supervisor and visualize the robot's state. Currently in alpha version, it runs on your local network and can be accessed from any device with a web browser. The backend is written in Python and uses the CGI protocol (Common Gateway Interface) to communicate with the web interface.

In order to get your account setup and use freely this RDM, please contact support@3lawsrobotics.com

This product is still at a alpha development stage. New features are developed every month and some breaking changes can still happened. The 3LawsRobotics Team is available to answer your question.

For now only a ROS1, ROS2 interfaces are configurable via this control panel.

4.1 Launch the Control Panel

The control panel is installed as a systemd service. To start the control panel, use the following command:

```
sudo systemctl start lll_control_panel
```

and to stop it:

```
sudo systemctl stop lll_control_panel
```

Also you can start it manually with the following command:

```
python3 /opt/3lawsRoboticsInc/control_panel/server.py
```

You can add the following command line options:

- **-p or –port**: <number, default: 8000> Specify the port to use

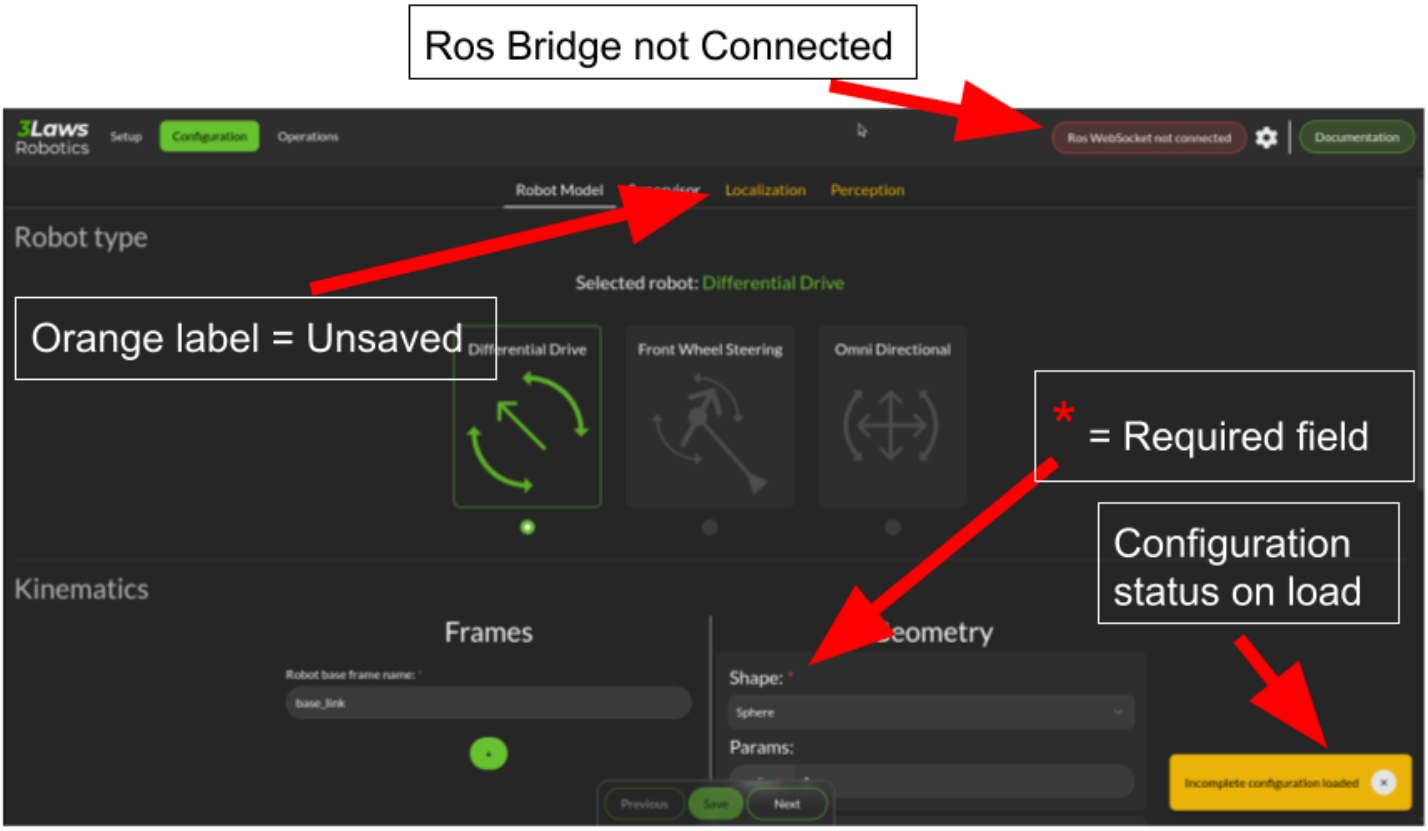
You can specify the emplacement of the configuration file by setting the environment variable *LLL_CONFIG_FOLDER*

4.2 Use the Control Panel

The control panel is meant to be user friendly and safe explanatory. However some detail might be

useful to know.

Here is a picture of the control panel:

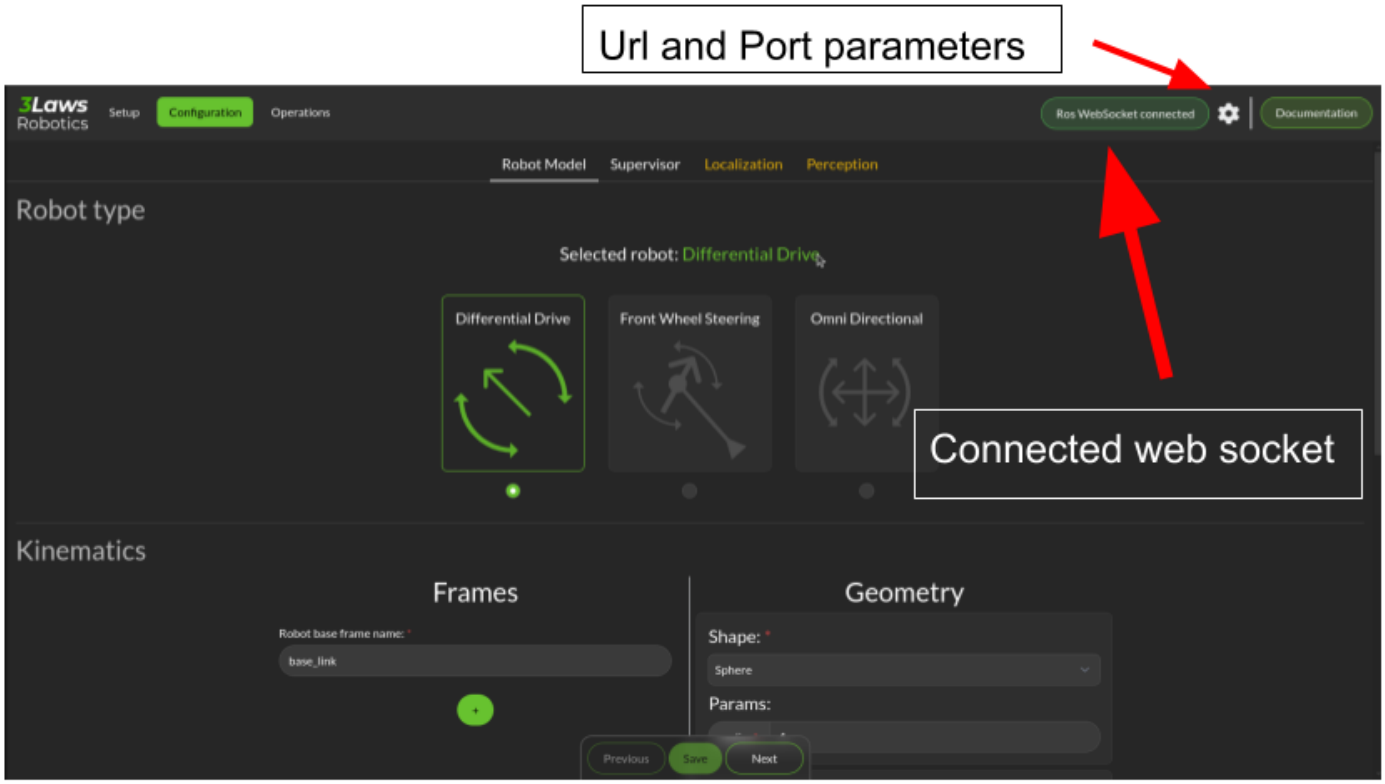


The control panel is divided in 3 parts:

- The Navigation bar, where you can find the main pages, a Ros Bridge WebSocket status indicator and a WebSocket parameter button.
- The main page, where you can find the configuration form and the control buttons.
- The footer, where you can find the version number and the 3LawsRobotics website link and a link to the update pages.

In order to use this control panel at its best, you need to configure the Ros Bridge WebSocket parameters. To do so, click on the WebSocket parameter button in the navigation bar and fill the form with the correct parameters.

By default, the control panel is configured to connect to a Ros Bridge WebSocket running on the same machine, on the port 9090. More information on the Ros Bridge WebSocket can be found following this link: Ros Bridge WebSocket.



A successful connection to the Ros Bridge WebSocket will be indicated by a green status indicator in the navigation bar.

Chapter 5

Visualization

Visualization is a key part of the monitoring system. The following sections will guide you through the visualization tools that are available to you.

Table of Contents

5.1 Dashboards 25
5.2 Navigation Tools 26
5.3 Incident Management 26

5.1 Dashboards

In order to visualize your monitoring data. 3LawsRobotics provides a Grafana dashboard. You can access it by opening your web browser and navigating to https://3laws.app. Your credentials will be given to you by your 3LawsRobotics representative.

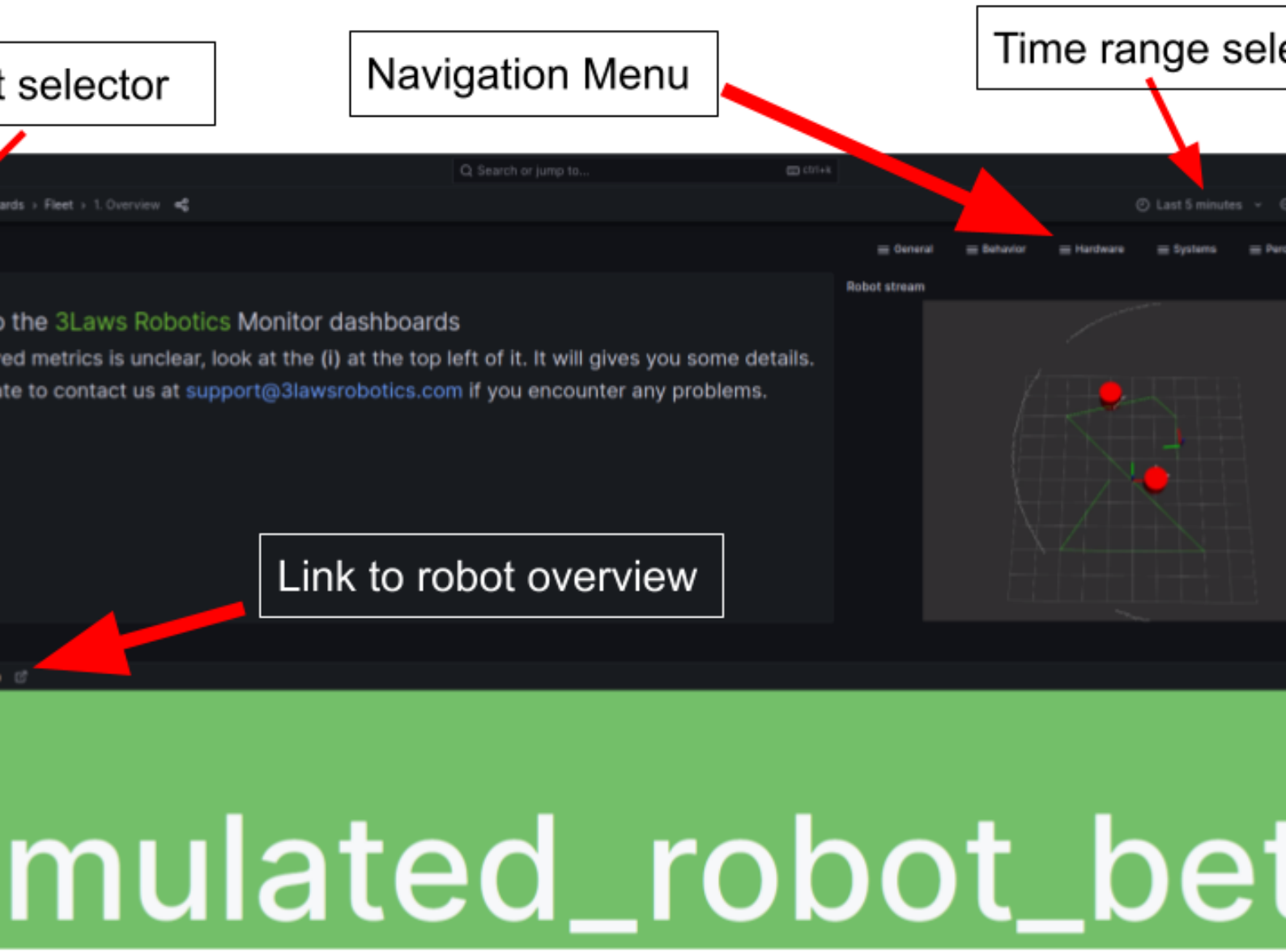
This dashboards are tailored to quick diagnostic. This means that the workflow will help you dive into the data that is relevant to your current problems. The dashboards are organized in the following way:

- **General**: This dashboard displays your robots that are currently monitored and their current state. It will also gives you a list of the latest events that have been recorded by the monitoring system.
- **Behavior**: This dashboard gives you insights into the behavior of your robot, like his computed safety score, your distance to obstacle, etc.
- **Hardware**: This dashboard expose the hardware health of the robot. It let you quickly see if the current physical behavior of the robot is coherent with your model or if some degradation is happening.
- **Systems**: This is the most important dashboard. It gives information about the system of the robot like signals delay, computer usage and Node status.
- **Perception**: This dashboard make it easy to diagnose a sensors issue like obstructed sensor or noisy sensor.
- **Control**: Finally this dashboard allows you to observe the good tracking and feasibility of the control commands or the planning algorithms

Each dashboard is composed of several panels with always an overview that helps you to quickly see if everything is fine and then more detailed panels that will help you to precisely diagnose the problem.

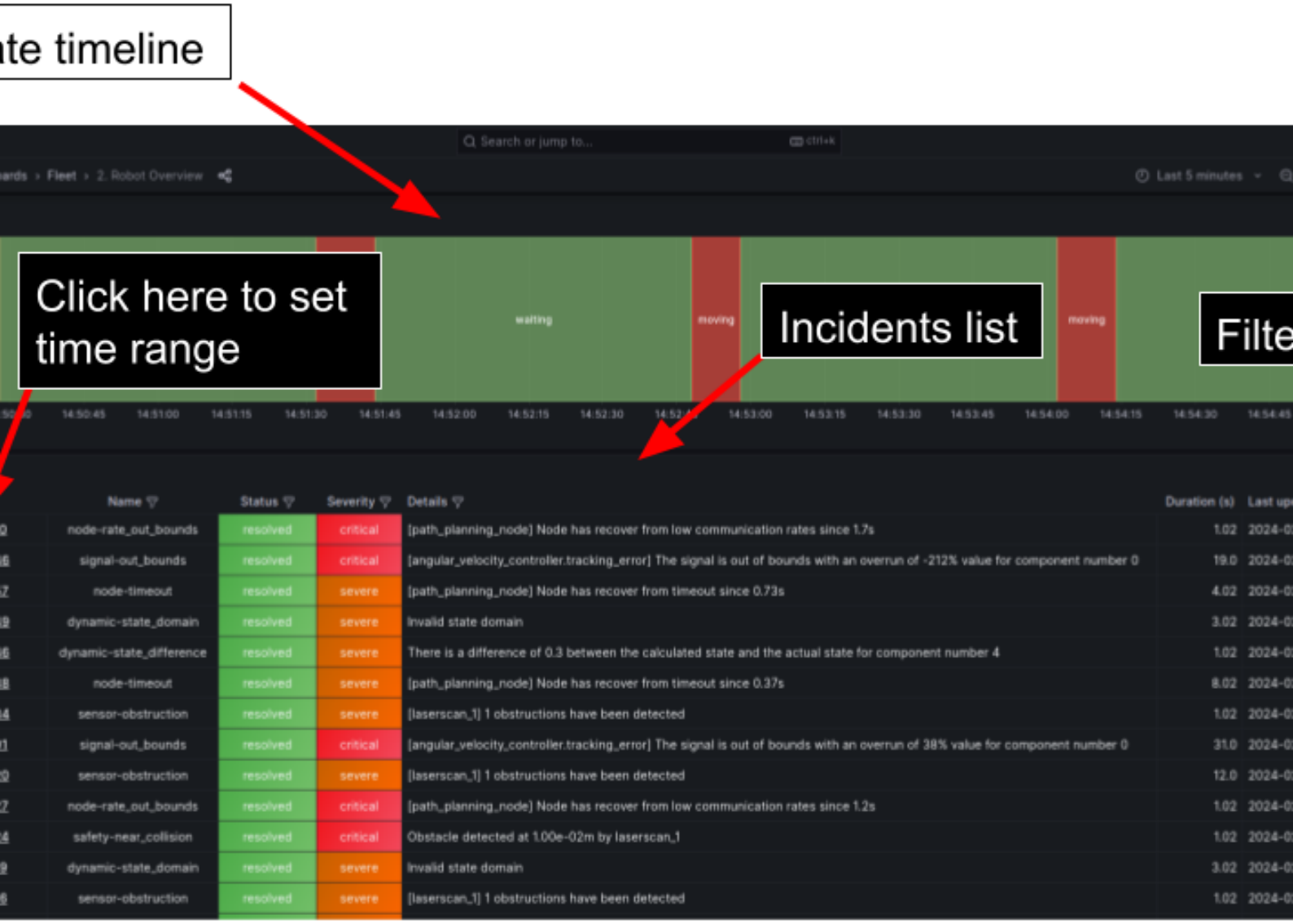
5.2 Navigation Tools

The dashboards are composed of several navigation tools that will help you to quickly diagnose a problem. Here is a screenshot of a dashboard with the navigation tools highlighted:



- **Time Range**: This tool allows you to select the time range of the data that you want to visualize. You can select a predefined range or a custom range.
- **Refresh**: This tool allows you to refresh the data displayed in the dashboard. You can select the refresh interval or manually refresh the data.

5.3 Incident Management

The Dashboards display a list of incident, you can use it to navigate directly to the right detailed dashboard. The timestamp of the incident is clickable and will bring to the right detailed dashboard at

the time of the incident.